

use Spark for batch.

A streaming aggregation in Spark looks like this.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import window, count

spark = SparkSession.builder.appName("EventCounts").
getOrCreate()
events = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "user_events") \
    .load()
counts = events \
    .groupBy(window("timestamp", "1 minute"), "event_type") \
    .agg(count("*").alias("count"))
query = counts.writeStream \
    .format("console") \
    .outputMode("update") \
    .start()
```

That's a streaming job. Read from Kafka, group by minute and event type, count, write somewhere. If you've used Spark before, this looks instantly familiar. If you haven't, the API is well documented and consistent with the batch version.

Spark Structured Streaming uses a micro-batch model by default, which means it's not strictly real-time. There's a small delay, typically a few seconds, between an event happening and it being processed. For most use cases, this is totally fine. If you need true sub-second latency, Flink is a better choice. For 'near real time' applications, Spark is excellent.

Debezium, the change data capture star

Most data engineering work involves moving data from operational databases to analytical systems, or between operational systems. Historically, this was done with periodic batch dumps. Take a snapshot of the database, copy it to the warehouse, repeat tomorrow.

Debezium changed the game by making change data capture, or CDC, easy. The idea is to tap into the database's transaction log and stream every change as it happens. Insert, update, delete, all of them become events flowing into Kafka in near real time. You can build downstream systems that react to these events without ever putting load on the source database.

Setting up Debezium connects to your database's replication slot and starts publishing changes to Kafka topics. Each topic corresponds to a table. Each event corresponds to a row change. The schema includes the before and after state, the operation type, and a bunch of metadata.

For teams trying to bridge the gap between operational and analytical workloads without rewriting their applications, Debezium is often the cleanest answer. It works with

PostgreSQL, MySQL, MongoDB, SQL Server, Oracle, and others. The setup is genuinely not bad. The Debezium documentation has reasonable Docker Compose examples that get you a working pipeline in an hour.

Apache Airflow, still the orchestrator most teams reach for

When you're firmly in batch territory, Airflow is the workflow orchestrator that most data teams end up using. It's been around since 2014, came out of Airbnb, and is now an Apache project with a huge community.

The core idea is that you write your workflows as Python code. You define directed acyclic graphs of tasks. Airflow schedules them, runs them, retries them on failure, and gives you a web UI to monitor everything. It's not glamorous, but it's solid, and it's saved an enormous amount of homegrown scheduler code over the years.

A simple Airflow DAG looks like this.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

def extract_data():
    pass

def transform_data():
    pass

def load_data():
    pass

dag = DAG(
    'daily_etl',
    start_date=datetime(2026, 1, 1),
    schedule_interval='@daily',
    catchup=False
)

extract = PythonOperator(task_id='extract', python_callable=extract_data, dag=dag)
transform = PythonOperator(task_id='transform', python_callable=transform_data, dag=dag)
load = PythonOperator(task_id='load', python_callable=load_data, dag=dag)
extract >> transform >> load
```

That's it. A daily ETL job with three stages and clear dependencies. Airflow handles the scheduling, the retries, the failure notifications. You write the business logic.

For pure streaming workloads, Airflow isn't the right tool. It's built around the idea of jobs that start and end. But for hybrid pipelines where streaming feeds into batch jobs for heavy aggregation, Airflow is often part of the picture.

There are newer alternatives worth knowing about. Dagster has a more modern Python API and a great UI. Prefect has interesting features around dynamic workflows.